

T2 – Algoritmos de Busca

Alocação de alunos em quartos duplos heterogêneos por Algoritmo Genético
Inteligência Artificial — Profa. Silvia Moraes — PUCRS
Integrantes: Vicente Piltcher, Michele Ughini e Ghabriel Girard

O problema e sua natureza

Alunos de duas escolas (A e B), mesmo tamanho N , em quartos duplos heterogêneos (exatamente um de cada escola por quarto).

Cada aluno respondeu um questionário → lista ordenada de preferência sobre a outra escola (do mais ao menos preferido).

Objetivo:

- emparelhamento que maximiza a satisfação mútua;

- restrições: todos alojados, nenhum quarto com dois da mesma escola.

Natureza:

- problema de atribuição (assignment), parente do Casamento Estável (Gale–Shapley);

- espaço de busca = $N!$ ($N=10 \rightarrow 3.628.800$) → exaustivo inviável.

Por que Algoritmo Genético Memético+ visão geral (pipeline)

AG: meta-heurística populacional inspirada na evolução (evolui um conjunto de soluções, combinando-as por gerações). Escolha permitida pelo enunciado (AG ou SA).

Memético = AG + busca local: combina exploração (AG) com intensificação (busca local) → convergência rápida e confiável ao ótimo.

Pipeline: 1° ler arquivo → 2° população inicial → 3° ciclo por geração (avaliar → elitismo → seleção → cruzamento → mutação → busca local → ordenar) → 4° parada → 5° saída.

Entrada e Codificação

Formato da entrada: linha 1 = N ; próximas N linhas = Escola A; próximas N = Escola B. Cada linha: id + lista ordenada de ids do outro lado (mais→menos preferido). A leitura valida que cada lista é uma permutação de $1..N$ (e trata linhas em branco).

Codificação (cromossomo): indivíduo = permutação perm, onde $\text{perm}[i]$ = aluno B alocado ao aluno A i . Toda permutação de $0..N-1$ é automaticamente um emparelhamento perfeito e heterogêneo (cada B usado exatamente uma vez).

Exemplo: 4 2 1 3 → A1-B4, A2-B2, A3-B1, A4-B3.

Função heurística / aptidão

Custo de insatisfação a MINIMIZAR: $\text{custo}(\text{perm}) = \sum_i [\text{rankA}(i, \text{perm}[i]) + 1] + [\text{rankB}(\text{perm}[i], i) + 1]$

$\text{rankA}/\text{rankB}$ = posição do par na lista (0 = 1ª escolha; o +1 deixa em base 1).

Mínimo por dupla = 2 (1ª escolha mútua); ótimo teórico global = $2 \cdot N$.

Exemplo (caso1): $A1-B4 = 3+1 = 4$; $A2-B2 = 1+2 = 3$; $A3-B1 = 2+1 = 3$; $A4-B3 = 1+2 = 3 \rightarrow$ solução 4 2 1 3 custa 13 (média 3,25/dupla).

Propriedades: simétrica (pondera A e B), independente de N (scale-free), monotônica (melhorar uma dupla nunca piora o total).

População, Seleção e Elitismo

População inicial: 100 permutações aleatórias válidas (`random.shuffle`) — equilíbrio entre diversidade/exploração e custo computacional.

Seleção por torneio ($k=2$): sorteia 2 indivíduos, vence o de menor custo. Define a pressão seletiva (k maior = mais elitista); não exige fitness normalizado.

Elitismo (2): os 2 melhores passam intactos para a próxima geração → a melhor solução nunca piora ao longo das gerações.

Cruzamento: Order Crossover (OX)

Taxa de cruzamento = 0,85 (com prob. 0,15 os filhos são cópias dos pais).

OX (operador clássico para permutações):

- 1) o filho herda um segmento contíguo $[a,b)$ de um pai;
- 2) completa as demais posições com os genes do outro pai, na ordem em que aparecem (ciclicamente, a partir de b), pulando os já herdados.

Preserva a permutação por construção → nunca repete ids, dispensa reparo.

Exemplo: $p1=[1\ 2\ 3\ 4\ 5\ 6]$, $p2=[4\ 1\ 5\ 2\ 6\ 3]$, segmento $[2,5)$ → herda $_ _ 3\ 4\ 5 _$, completa com $1\ 2\ 6$ → $2\ 6\ 3\ 4\ 5\ 1$ (gera 2 filhos por casal).

Mutação e Busca local (memético)

Mutação por troca (swap), taxa 0,15:

- troca duas posições da permutação;
- preserva a validade;
- mantém diversidade e ajuda a escapar de ótimos locais.

Busca local 2-swap (first-improvement):

- após cada filho, testa trocar pares de posições e aplica imediatamente qualquer troca que reduza o custo, até nenhum swap melhorar (ótimo local da vizinhança 2-swap).

Delta O(1) (não recalcula o fitness inteiro): $\text{delta} = (\text{custo}[a][jb] + \text{custo}[b][ja]) - (\text{custo}[a][ja] + \text{custo}[b][jb])$.
É o que leva o AG ao ótimo de forma confiável (componente memético).

Ciclo do AG e critérios de parada

Ciclo (por geração): avaliar população → elitismo → seleção (torneio) → cruzamento (OX) → mutação (swap) → busca local → ordenar por custo → repetir.

Critérios de parada (o que ocorrer primeiro):

- 1) ótimo teórico atingido ($\text{custo} \leq 2 \cdot N$);
- 2) máximo de gerações = 500;
- 3) estagnação = 50 gerações sem melhora do melhor custo.

O melhor global é guardado a cada geração (nunca se perde).

Parâmetros utilizados

Parâmetro	Valor	Efeito de aumentar / justificativa
População	100	mais diversidade/exploração; mais custo por geração
Gerações (máx.)	500	mais refino; risco de desperdício
Crossover (OX)	0,85	mais recombinação (exploração)
Mutação (swap)	0,15	mais diversidade; alta demais aleatório
Torneio (k)	2	mais pressão seletiva (converge mais rápido)
Elitismo	2	mais estabilidade; alto demais reduz diversidade
Estagnação	50	parar após convergir
Busca local	ligada	intensificação (memético)

Modos de execução e saída

Passo a passo (--modo passo): pausa a cada geração (Enter) e mostra o melhor indivíduo codificado e decodificado.

Final (--modo final): roda tudo e exibe a solução ao final.

Ambos imprimem a evolução da heurística (melhor/média/pior por geração).

Saída decodificada (exemplo): Quarto 1: A1 <-> B4 (A escolheu B como 3a opcao; B escolheu A como 1a opcao; custo=4)

Persistência: console + arquivo <arquivo>_output.txt (solução + log completo).

Resultados e impacto do memético

N=4 (caso1.txt): custo 13 = ótimo (confirmado por força bruta nas 24 permutações) → solução 4 2 1 3.

N=10 (caso2.txt): solução válida, custo 30 (média 3,0/dupla); ótimo teórico 20 não atingível (preferências conflitantes) — 30 é a melhor solução real.

AG puro x memético: sem busca local, o AG "puro" estaciona em custos piores nas mesmas gerações → a busca local é decisiva.

Problemas e considerações

Cruzamento: corte simples quebrava permutações (exigia reparo + penalidade) → migramos para Order Crossover, que preserva validade e herda mais estrutura.

Penalidade removida: com OX + swap + busca local, soluções são sempre válidas.

Calibração de taxas: mutação alta demais vira busca aleatória; torneio grande demais converge cedo (ótimo local).

Heurística simétrica foi decisiva para soluções justas entre as duas escolas.

Estagnação evita desperdício após a convergência; matriz de custo garante eficiência (avaliação $O(N)$, delta $O(1)$).

Encerramento

Obrigado!